

Tool-Agnosticism, Authority-Not-Labor, Linear-Cost: The Three Architectural Commitments Producing Governance Accessibility in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 5, 2026

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize, as a standalone derivation, the joint specification by which three previously-established CKS commitments — tool-agnosticism (§7.1), authority-not-labor (§2.1, §2.3), and linear-cost scaling (§6.3) — compose to produce the non-specialist governance accessibility committed to in §7.4 of the source paper.

Abstract

The CKS pattern commits, in §7.4, to non-specialist governance as an architectural property: the operational accessibility under which non-specialists can exercise governance over substrate content using tools they already use, without authorship expertise, and without cost penalties that grow with substrate size. That commitment is not free-standing; it is produced by three architectural commitments operating jointly — tool-agnosticism (the substrate's three minimal requirements are satisfiable by commodity tools), authority-not-labor (the three rights are held over substrate content regardless of authorship), and linear-cost scaling (governance cost does not scale with substrate size). This note formalizes the three-commitments specification as standalone: each commitment's contribution through a specific architectural mechanism, the joint composition that produces accessibility, the necessity of each commitment, the architectural-pattern-level operation, four adjacent patterns commonly conflated with it, ten failure modes, and an operational test for instantiation.

1. Why the joint specification needs to be formalized as standalone

A1.11 commits to non-specialist governance as an architectural property of the CKS pattern. A2.64 enumerated the three architectural commitments producing accessibility at the integrating-frame level; A2.65 formalized the governance-versus-authorship distinction; A2.66 formalized the four authorship roles. What remains is the *governance-side mechanism* — the precise architectural means by which governance is rendered accessible. This note supplies that derivation.

Without a precise specification, accessibility appears either as a vague architectural property without a producing structure, or as a deployment-level feature without an architectural commitment. The standalone treatment makes the production mechanism specific: the architecture commits to three commitments

composing in a particular way, and the composition is what produces accessibility. The strategic prior-art posture is the second motivation: patentable derivations focused on accessible AI architectures, non-specialist AI governance systems, or commodity-tool AI infrastructure are more defensibly contested when the production mechanism is publicly formalized as standalone. A1.05, A1.01, and A1.06 each have their own decomposition; this note does not relitigate any of them.

2. Tool-agnosticism's contribution to accessibility

Tool-agnosticism per A1.05 commits to the substrate's three minimal requirements (per A2.24–A2.26): persistent structured state, human read/write access, and LLM access to substrate content. The architectural commitment is that any environment satisfying these three requirements can host the substrate, and the requirements are satisfiable by commodity tools (file systems, document repositories, spreadsheet applications, version control systems, web interfaces) that non-specialists already use. Specialist platforms are not architecturally required. The contribution to accessibility follows: the inspect right per A2.01 is exercised by reading substrate content in the hosting tool, the modify right per A2.02 by writing in it, and the override right per A2.03 by direct edit. The governance interface is the tool the non-specialist already uses; no learning curve for a specialist platform is architecturally required. The architectural barrier this commitment removes is *platform-bound governance* — the barrier under which governance is accessible only to those who can use a specialized platform, regardless of how willing they are to exercise the rights.

3. Authority-not-labor's contribution to accessibility

Authority-not-labor per A1.01 commits to authority being the architectural object humans govern, with the three rights (inspect, modify, override) being the actions humans take. The architectural commitment is that authority is held over substrate content regardless of who or what authored that content; authorship is not a precondition for governance authority. The contribution to accessibility is that non-specialists exercise authority without needing to have authored the content and without needing authoring expertise: a non-specialist who has not authored any of the four authorship roles per A2.66 — schema, rule, cell, or initial content — still holds the three rights per A2.01–A2.03. The governance-versus-authorship distinction per A2.65 is what makes this true at the action level; A1.01 is what makes it true at the architectural-commitment level. The architectural barrier this commitment removes is *authorship-required governance* — the barrier under which governance is accessible only to those who could have authored the content they govern.

4. Linear-cost's contribution to accessibility

Linear-cost per A1.06 commits, among other cost properties, to governance cost as not size-proportional (per A2.33). The architectural commitment is that the cost of exercising governance scales with intervention frequency — a deployment choice — not with substrate size; a non-specialist governing a small substrate and a non-specialist governing a large substrate face the same cost structure. Inspect operations per A2.01 are bounded by what the non-specialist chooses to inspect, not by total substrate volume; modify and override per A2.02–A2.03 are bounded by chosen intervention frequency, not by substrate size. The architectural barrier this commitment removes is *size-proportional cost*. Without the commitment, governance accessibility would degrade with scale: a substrate small enough to be governed cheaply by a non-specialist could grow to a size where governance cost exceeded what the non-specialist could pay, even with the inspect, modify, and override rights nominally available. Linear-cost preserves accessibility through scale rather than only at small scale, which is what makes accessibility architecturally meaningful rather than scale-conditional.

5. Three further operational components of the joint specification

The contributions in §§2–4 are operationally distinct, but the joint specification is more than their sum. It has three further components.

(a) The joint composition. The three commitments compose architecturally to produce accessibility no commitment alone could produce. Tool-agnosticism alone yields platform flexibility; authority-not-labor alone yields authority decoupling; linear-cost alone yields cost stability — none yields accessibility, because each leaves at least one of the other two architectural barriers in place. The composition is *not feature-additive*: stacking the commitments as separate features composed by aggregation does not produce accessibility. Each commitment is itself architectural; the accessibility produced by their joint operation is itself architectural.

(b) The necessity of each commitment. Each commitment is necessary. Removing tool-agnosticism produces platform-bound governance; removing authority-not-labor produces authorship-required governance; removing linear-cost produces governance scaling super-linearly with substrate size. None is redundant; none is sufficient alone. Partial composition produces partial accessibility — a failure mode rather than a graceful degradation.

(c) Architectural-pattern-level operation. The specification operates at the architectural-pattern level. The architectural commitment is to the three commitments composing per their own architectural content, independent of deployment-level enhancement. Deployments may add interfaces, training, or support that operationally enhance accessibility, but architectural accessibility is what the three commitments compose to produce, not what those deployment choices produce.

6. What the joint specification does NOT claim

Six limitations earn explicit naming.

It does not claim that all governance actions are equally simple. Specific actions may still require domain understanding (overriding a decision per A2.03 may require understanding the decision's context); substantive understanding is a separate concern from architectural accessibility.

It does not specify deployment-level accessibility features. Deployments may have specific simplified interfaces, training programs, or support resources; the architectural commitment is to the three commitments themselves, not to deployment-level features that complement them.

It does not claim outcome equivalence between specialist and non-specialist governance. Non-specialists may exercise governance with different intervention frequencies, override decisions, or inspection patterns. The commitment is that the rights are exercisable, not that outcomes are equivalent.

It does not foreclose specialist governance. Deployments may have specialists exercising governance alongside or instead of non-specialists; the commitment is that non-specialist governance is feasible, not that all governance must be non-specialist.

It does not require deployments to use commodity tools. Tool-agnosticism per A1.05 commits to the three minimal requirements being satisfiable by commodity tools; deployments that prefer specialist platforms may use them. The commitment is to the *option* of commodity-tool governance.

It does not specify implementation. Implementations may instantiate each commitment operationally in various ways; the commitment is to the three commitments being satisfied per their decompositions.

7. What the joint specification is NOT — four adjacent patterns

The specification is commonly conflated with four adjacent accessibility patterns. Each is reasonable in some other architecture; conflating any with the architectural commitment misreads what the architecture produces.

Not usability features. Intuitive interfaces, friendly error messages, and helpful documentation operationally improve accessibility but are not what the architectural commitment names. Implementations may have excellent usability without satisfying the three commitments, or poor usability while satisfying them. The commitment is at the architectural-pattern level, not the interface-quality level.

Not training programs. Training reduces the operational expertise gap between specialists and non-specialists. The architectural commitment is different in kind: accessibility is produced by the three commitments operating, not by training closing a gap the architecture leaves open. A system requiring extensive training fails the commitment regardless of the training's quality.

Not simplified interfaces. Curated dashboards, guided workflows, and reduced feature surfaces operationally enhance accessibility for non-specialists but are deployment-level constructions that may complement the three commitments without substituting for them.

Not vendor support tiers. Customer support, professional services, and managed deployment operationally enhance accessibility by interposing expert assistance. A system whose accessibility depends on vendor support is operationally accessible but architecturally inaccessible — the support could be removed, repriced, or deprecated, and the architecture's accessibility property would not survive the change.

8. Why the joint specification is load-bearing

The joint specification is load-bearing for several CKS commitments. For A1.11 and A2.64, the three commitments composing are what produces the architectural property; without them, A1.11 would be aspirational rather than architecturally produced. For A2.65, the three commitments are what makes the governance-versus-authorship distinction operationally meaningful for accessibility — without them, governance would be inaccessible regardless of how cleanly the boundary were drawn. For A2.66, the three commitments are what produce governance accessibility despite the existence of authorship requirements; without them, accessibility would depend on authorship expertise and the distinction would collapse at the accessibility layer. For the labor allocation framework per A1.12, the joint specification is what makes governance accessibility *mode-independent*: authority-not-labor ensures authority is the same across the three production modes, tool-agnosticism ensures the host environment is the same across modes, linear-cost ensures cost properties are the same across modes. Finally, for A2.53 (CKS $\neq$ OIDA + multi-human), the joint specification articulates how the architectural difference — which rests in part on tool-agnosticism and authority-as-substrate-content — produces operational accessibility specifically.

9. Failure modes — ten ways to miss the specification

Each failure mode names a way an implementation can fail the architectural commitment by missing or operationally compromising one or more of the three commitments.

(a) Missing tool-agnosticism. The implementation requires a specialist platform — a custom dashboard, vendor-specific application, or enterprise governance suite — and inspect, modify, and override are nominally available but only inside the platform.

(b) Missing authority-not-labor. The implementation treats governance as requiring authorship expertise; non-specialists are blocked from exercising rights they architecturally hold per A1.01 because the implementation requires schema, rule, cell, or initial-content authoring expertise to exercise modify or override.

(c) Missing linear-cost. Governance cost scales super-linearly with substrate size — through inspection mechanisms that load the entire substrate, modification mechanisms that require search across it, or override mechanisms whose latency grows with content volume — and non-specialists face cost prohibitions on large substrates.

(d) Missing two or three commitments. Implementations missing multiple commitments produce architectures with multiple barriers; non-specialist governance is inaccessible along multiple dimensions, and the system's claim to architectural accessibility cannot be defended along any of the missing dimensions.

(e) Treating commitments as feature-additive. The implementation adds tool-agnosticism, authority-not-labor, and linear-cost as separate features composed by feature aggregation rather than by architecture. The architectural composition per §5(a) is broken; the commitments do not produce accessibility architecturally because their composition is not architectural.

(f) Treating commitments as deployment options. The implementation makes the three commitments configurable through deployment options or runtime flags. The architectural commitment is broken; the

commitments become deployment choices, and deployments may run without one or more of them while still being labeled as instantiations of the pattern.

(g) Operationally simulating accessibility without architectural commitments. The implementation provides simplified interfaces, training programs, or vendor support that operationally enhances accessibility for non-specialists, but the architectural commitments are absent. Accessibility is operational, not architectural; it depends on deployment-level features that may be removed, repriced, or degraded over the deployment's lifetime, after which the architectural absence is exposed.

(h) Tool-agnosticism without commodity-tool capacity. The implementation claims tool-agnosticism but the substrate requires schemas, rule formats, or cell behaviors so complex that no commodity tool can practically support them. Tool-agnosticism is technically present but operationally compromised; the three minimal requirements per A2.24–A2.26 are nominally satisfied but practically violated.

(i) Authority-not-labor with hidden labor requirements. The implementation claims authority-not-labor but the operational interfaces for exercising the rights require authorship-equivalent expertise to navigate (the modify interface requires understanding rule expression formats; the override interface requires understanding cell-internal state; the inspect interface requires schema knowledge to interpret content). Authority-not-labor is operationally compromised by interface design that smuggles authorship requirements back in.

(j) Linear-cost in name only. The implementation claims linear-cost but governance cost in practice scales super-linearly with substrate size due to operational mechanisms not visible in the architectural commitment — inspection requires loading the entire substrate, override requires search across it, modify requires propagation through dependencies that grow with size. The cost contract per A1.06 is nominally honored at the architectural level but operationally violated.

10. Operational test

A system instantiates the three-commitments specification if and only if all of the following are true.

1. Tool-agnosticism per A1.05 is operationally satisfied — the three minimal requirements per A2.24–A2.26 are met, and non-specialists can in fact govern through commodity tools.
2. Authority-not-labor per A1.01 is operationally satisfied — humans hold the three rights per A2.01–A2.03 over substrate content regardless of authorship, and non-specialists exercise them without authorship expertise.
3. Linear-cost per A1.06 is operationally satisfied — governance cost is not size-proportional per A2.33, and non-specialists can govern substrates of any size with cost bounded by intervention frequency rather than substrate volume.
4. The three commitments compose architecturally per §5(a); the composition is not feature-additive.
5. Each of the three commitments is necessary per §5(b); removing any one would produce an architectural barrier.
6. The specification operates at the architectural-pattern level per §5(c); deployment-level features may complement but do not substitute for the commitments.

A system that fails any of (1)–(6) does not instantiate the joint specification in the architectural sense, even if its operational interfaces appear accessible to non-specialists.

11. Why naming the joint specification matters

Implementations under pressure to deliver accessible AI architectures consistently drift toward operational-accessibility patterns that do not produce architectural accessibility. The drift is steady because operational features are commercially familiar and rhetorically tractable: audiences understand "we have a friendly interface" or "we provide white-glove onboarding" more easily than "the architecture has three commitments that compose to produce accessibility." Implementations that drift produce systems where accessibility is operationally claimed but architecturally absent — with predictable downstream consequences: accessibility-feature degradation when deployment features are removed; specialist-platform lock-in once a training period ends; governance-failure-at-scale when a small-scale cost structure fails; and architectural-difference weakness in the sense of A2.53.

Naming the joint specification as a standalone architectural commitment gives downstream readers a precise specification of what produces governance accessibility architecturally. With this note, the four-note non-specialist-governance decomposition is complete: A2.64 supplied the integrating frame, A2.65 the action-level boundary, A2.66 the authorship-side roles, and this note the governance-side mechanism. Subsequent work that uses "non-specialist governance accessibility" differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Tool-Agnosticism, Authority-Not-Labor, Linear-Cost: The Three Architectural Commitments Producing Governance Accessibility in the Coordination Knowledge Substrate Pattern*. May 5, 2026. ORCID: 0009-0004-8065-3235.